

OOP Mastery

Object-oriented programming for the placement round

four pillars, polymorphism, SOLID

8 diagrams, 19 model answers, 45 FAQs

[@rathoreadiitya](#)

v1.0 · Updated 2026-06-06

Brand new to OOP? Start at [§3 Start here: the student minimum](#) below, then read the core concepts in order.

This sheet is Java-first: every code sample is Java, because that is what most OOP rounds use. Wherever C++ behaves differently you get a peach 'C++ differs' box right next to it, so you always know which language a rule belongs to. The concepts are the same in any object-oriented language; only the syntax and a few rules change.

The four places Java and C++ genuinely diverge - memorise these so you never mix them up:
(1) access modifiers - Java has 4 levels incl. package-private, C++ has 3 and no package idea (§11); **(2) multiple inheritance** - Java bans it for classes, C++ allows it (§7); **(3) virtual methods** - every Java instance method is virtual by default, C++ needs the 'virtual' keyword (§9); **(4) memory** - Java has a garbage collector and no destructors, C++ has manual new/delete and ~destructors (§6). Everywhere else, assume the Java answer unless a C++ box says otherwise.

Inside this sheet

1. Brand-new note + who this sheet is for
2. Inside this sheet (you are here)
3. Start here: the student minimum - what an OOP round really tests
4. What OOP is + class vs object
5. The four pillars (with the confusion-killer)
6. Constructors, this, super, destructors
7. Inheritance types + the diamond problem
8. Polymorphism - overloading vs overriding
9. Dynamic dispatch + the vtable
10. Abstract class vs interface
11. Access modifiers
12. Association vs aggregation vs composition
13. Static vs dynamic binding + static members
14. Shallow copy vs deep copy
15. Exception handling essentials (Java)
16. Java Collections crash course
17. Why String is immutable + the String pool
18. Beyond the basics (advanced, not all fresher-required)
19. SOLID principles
20. Design patterns (recognise the words)
21. Java vs C++ - the OOP differences
22. Comparison tables (the ones interviewers love)
23. Interview triggers - if they ask X, say Y
24. Common mistakes + traps

- 25. Model interview answers - say it like this (19 questions)
- 26. Rapid-fire FAQ bank - 45 quick-recall questions
- 27. The 30-minute revision card
- 28. Final word

If you have only 30 minutes - jump to §27 Revision card.

The whole sheet on one screen (mind map)

How every topic hangs together. Glance at this before you dive in, and again the night before, so you walk in knowing the shape of the whole subject, not a pile of loose definitions.

OOP interview mind map

```
OOP
|
|- Four pillars
|   |- Encapsulation    (private + getters/setters)
|   |- Abstraction      (abstract class / interface)
|   |- Inheritance      (is-a, code reuse)
|   \- Polymorphism     (one name, many forms)
|
|- Inheritance
|   |- Types            (single / multilevel / hierarchical / multiple)
|   \- Diamond problem  (Java bans multi-class; C++ virtual)
|
|- Polymorphism
|   |- Overloading      (compile-time, same class)
|   \- Overriding       (runtime, vtable, subclass)
|
|- Abstraction
|   |- Abstract class   (shared code + state, extend ONE)
|   \- Interface        (a capability, implement MANY)
|
|- Relationships
|   |- Association      (uses-a)
|   |- Aggregation      (weak has-a, child survives)
|   \- Composition      (strong has-a, child dies with parent)
|
|- Java essentials      (exceptions, collections, String pool)
|
\ - SOLID + design       (5 principles, patterns, cohesion/coupling)
```

3. Start here: the student minimum

An OOP round for a fresher is a small core asked over and over. Learn this core cold and you will not drop easy points in most rounds.

The five must-knows:

1. The four pillars - encapsulation, abstraction, inheritance, polymorphism - with a one-line real example each, AND the difference between abstraction and encapsulation (the single most-asked trap).
2. Overloading vs overriding - which is compile-time and which is runtime, with a tiny code example.
3. Abstract class vs interface - what each can hold and when to pick which.
4. Inheritance types + the diamond problem - and how Java vs C++ handle it.
5. Association vs aggregation vs composition - the has-a strengths, with the car-engine and department-student examples.

The one hands-on thing to drill: write a tiny class hierarchy (Animal with a `speak()` method, Dog and Cat overriding it) and call `speak()` through an Animal reference. Watching the right version run is runtime polymorphism in one screen. If you can write it cold, you understand the core.

Where to jump: the pillars are §5, polymorphism is §8, abstract vs interface is §10, the has-a relationships are §12. Everything before §18 is the fresher core. SOLID and design patterns after it are depth for senior rounds.

4. What OOP is + class vs object

Object-oriented programming models a program as objects that bundle data (fields) with the behaviour that acts on it (methods), instead of loose functions passing data around. A class is the blueprint; an object is one built instance of that blueprint.

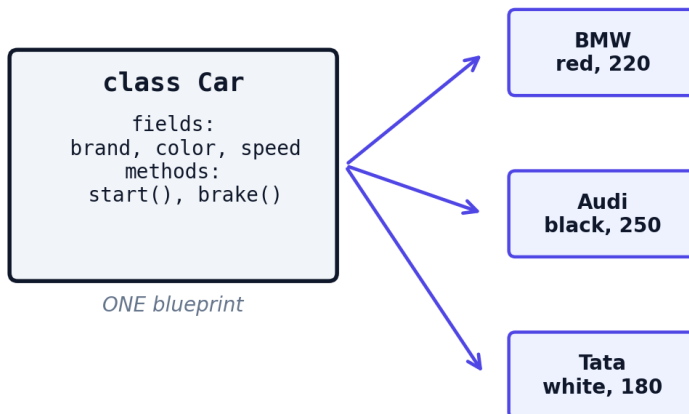
Java

```
class Car {                                // the blueprint
    String model;                          // field (state)
    void drive() {                         // method (behaviour)
        System.out.println(model + " is driving");
    }
}

Car c = new Car();                        // c is an object (an instance)
c.model = "Swift";
c.drive();                                // Swift is driving
```

Class = blueprint (written once) ; Objects = instances (made many)

new Car(...) -> many objects, each with its own state



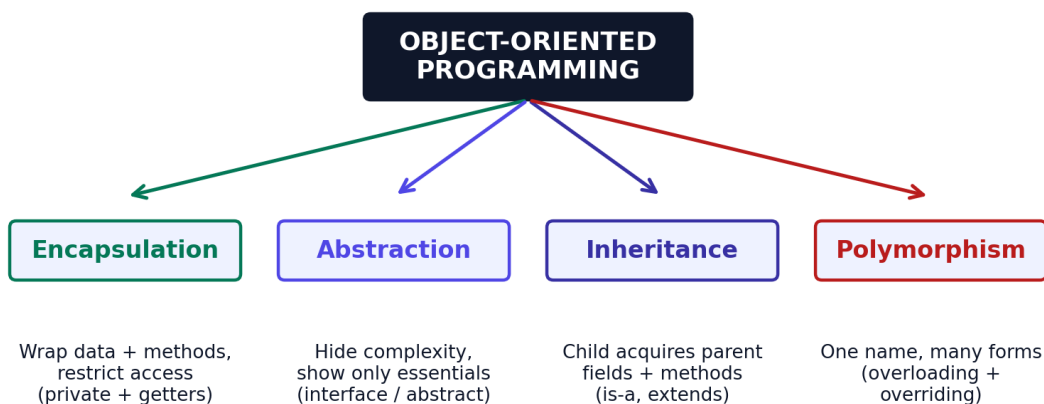
One class (blueprint) produces many objects (instances), each with its own field values.

💡 Aha - Class ek cookie cutter hai, object woh actual cookies jo usse bante hain. Cutter ek hai par cookies kai - har cookie apne flavour (field values) ke saath, par shape (structure) sabka same.

5. The four pillars (with the confusion-killer)

Every OOP round starts here. Know each pillar as a one-line idea plus a real example.

The 4 pillars of OOP (everything else is built on these)



The four pillars: encapsulation, abstraction, inheritance, polymorphism.

Encapsulation - bundle + protect

Wrap data and the methods that use it into one unit, and control access (make fields private, expose getters and setters). It stops outside code from putting the object in a bad state.

💡 **Aha - Encapsulation matlab medicine ka capsule - dawai andar band, tum seedha powder ko haath nahi lagate, capsule (getter/setter) ke through hi lete ho. Galat dose se bach gaye.**

Abstraction - hide complexity, show essentials

Expose only what a caller needs and hide how it works. A car's steering and pedals are the interface; the engine internals are hidden. Achieved with abstract classes and interfaces.

💡 **Aha - Abstraction matlab car chalana - tum steering aur brake jaante ho, engine ke andar kya ho raha hai jaanne ki zaroorat nahi. Bas zaroori cheez dikhi, complexity chhupi.**

The confusion-killer (most-asked OOP trap): encapsulation is about **HOW** you bundle and protect data (implementation level, done with private + getters and setters). Abstraction is about **WHAT** you expose versus hide (design level, done with abstract classes and interfaces). Encapsulation hides the data; abstraction hides the complexity. Different concerns - never say they are the same.

Inheritance - reuse via is-a

A child class derives fields and methods from a parent and can add or change behaviour. A Dog is-a Animal, so Dog reuses Animal's code. It models an is-a relationship.

💡 **Aha - Inheritance matlab parivaar ke gun - jaise bachche ko parents se kuch features mil jaate hain, waise child class ko parent class ke fields aur methods mil jaate hain, aur apne naye bhi add kar sakti hai.**

Polymorphism - one name, many forms

The same method name behaves differently depending on the object or arguments. A speak() call gives a bark on a Dog and a meow on a Cat. Two flavours: compile-time (overloading) and runtime (overriding).

💡 **Aha - Polymorphism matlab ek hi shabd, alag alag matlab object ke hisaab se. 'Cut' bolo barber ko toh baal, doctor ko toh operation. Same naam, alag kaam - actual object decide karta hai kaunsa version chalega.**

6. Constructors, this, super, destructors

- Constructor - a special method that runs when an object is created, to initialise its fields. Same name as the class, no return type.

- Default vs parameterised - the compiler gives a no-arg default if you write none; add your own to take values.
- this - refers to the current object; use it to tell a field apart from a parameter of the same name.
- super - refers to the parent; super() calls the parent constructor, super.method() calls the parent's version.
- Constructor chaining - this(args) calls another constructor in the SAME class (must be the first line), so you write the init logic once.
- Destructor / cleanup - C++ has destructors that run when an object dies. Java has no destructors; the garbage collector frees memory, and you use try-with-resources or close() for resources.

C++ differs: C++ has explicit destructors (~ClassName) and you often manage memory by hand (new / delete). Java leaves memory to the garbage collector, so there is no destructor to write.

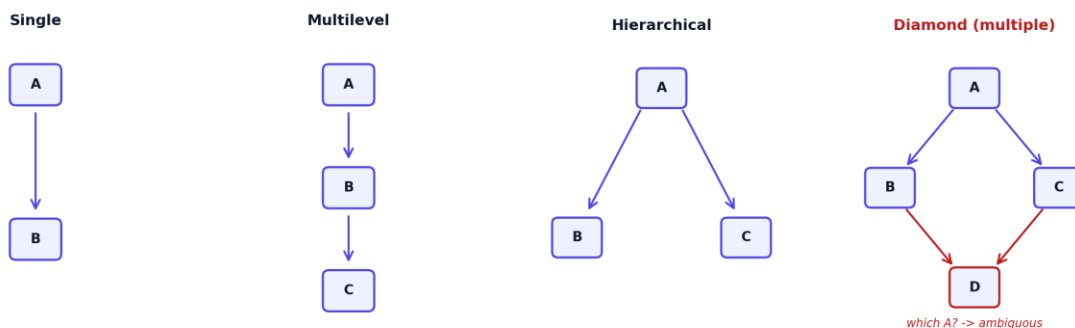
💡 Aha - Constructor woh pehla din hai jab object banta hai - naam, age, sab set ho jaata hai. 'this' matlab 'main khud', aur 'super' matlab 'mere papa (parent class)' - super() se parent ka constructor chal jaata hai pehle.

7. Inheritance types + the diamond problem

The common inheritance shapes:

- Single - one child, one parent (B extends A).
- Multilevel - a chain (C extends B extends A).
- Hierarchical - many children share one parent (Dog and Cat extend Animal).
- Multiple - one child, many parents. Java does NOT allow this for classes; you simulate it by implementing multiple interfaces. C++ allows it directly.
- Hybrid - a mix of the above.

Inheritance types (Java allows multiple only via interfaces)



Single, multilevel, hierarchical, and multiple inheritance shapes.

The diamond problem

If class D inherits from B and C, and both B and C inherit from A, then D could get two copies of A's members - which one wins? That ambiguity is the diamond problem. C++ solves it with virtual inheritance so only one shared A exists.

Java sidesteps it by banning multiple class inheritance; if two interfaces give the same default method, the class MUST override it to resolve the clash (and can call `Interface.super.method()`).

C++ differs: C++ allows multiple class inheritance, so the diamond is real there; you fix it with 'virtual' inheritance. Java forbids multiple class inheritance entirely, so the diamond only appears with conflicting interface default methods.

💡 **Aha - Diamond problem matlab do raaste ek hi dada tak jaate hain, ab pota confuse ki dada ki property kaunse raaste se le. Java bolta hai do baap (classes) allowed hi nahi, jhagda khatam. C++ bolta hai ek hi dada maano (virtual inheritance).**

final - the keyword that blocks change

Interviewers ask what final does at each level. It blocks change in three places:

- final class - cannot be subclassed (String, Integer are final).
- final method - cannot be overridden, even in a non-final class.
- final variable - assigned once, cannot be reassigned (a constant).

💡 **Aha - final matlab 'ab isme koi change nahi'. final class pe koi inherit nahi kar sakta, final method ko koi override nahi kar sakta, final variable ek baar set hua toh badal nahi sakte.**

8. Polymorphism - overloading vs overriding

Two methods with the same name, two completely different mechanisms. Mixing them up is a classic interview slip.

Overloading - compile-time (static)

Same method name, different parameter lists (number, type, or order), in the SAME class. The compiler picks which one to call from the argument types. No inheritance needed.

Java

```
class Printer {
    void print(int x)    { System.out.println("int: " + x); }
    void print(String s) { System.out.println("str: " + s); }
}
// print(5) and print("hi") resolved at COMPILE time
```

Overriding - runtime (dynamic)

A subclass redefines a parent's instance method with the SAME signature. The JVM picks the version based on the actual object at runtime, not the reference type. This is what gives true runtime polymorphism.

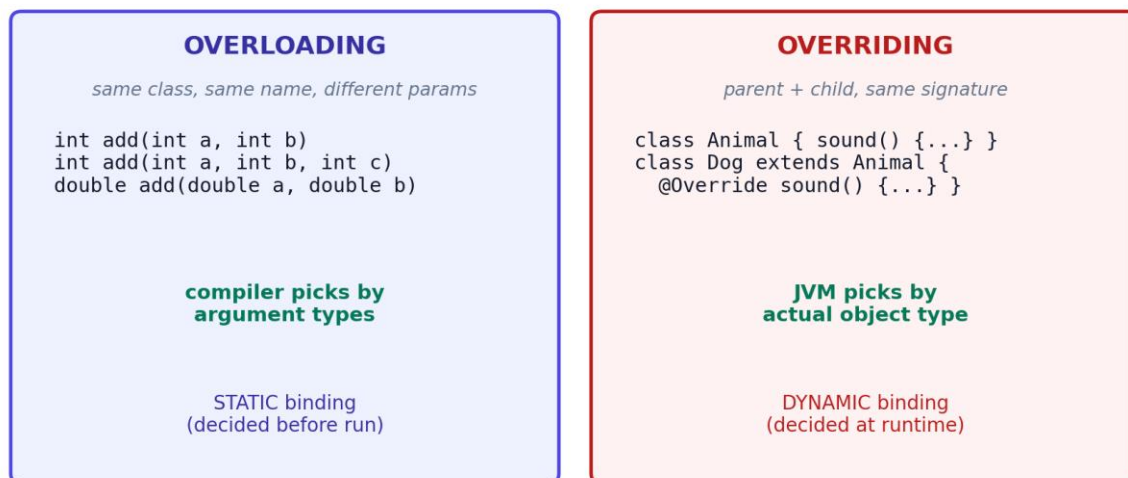
Java

```

class Animal { void speak() { System.out.println("..."); } }
class Dog extends Animal {
    @Override void speak() { System.out.println("Bark"); }
}
Animal a = new Dog();
a.speak();    // Bark - chosen at RUNTIME by the real object

```

Overloading (compile-time) vs Overriding (runtime)



Overloading is resolved at compile time by argument types; overriding is resolved at runtime by the actual object.

C++ differs: In Java every instance method is virtual by default, so overriding just works. In C++ the parent method must be marked 'virtual' or the call binds statically to the reference type and you do NOT get runtime dispatch.

💡 **Aha - Overloading compile-time pe decide hota hai - same naam, alag arguments, jaise ek hi dukaan pe alag size ke kapde. Overriding runtime pe - child class parent ka method badal deti hai, aur jo actual object hai uska version chalta hai.**

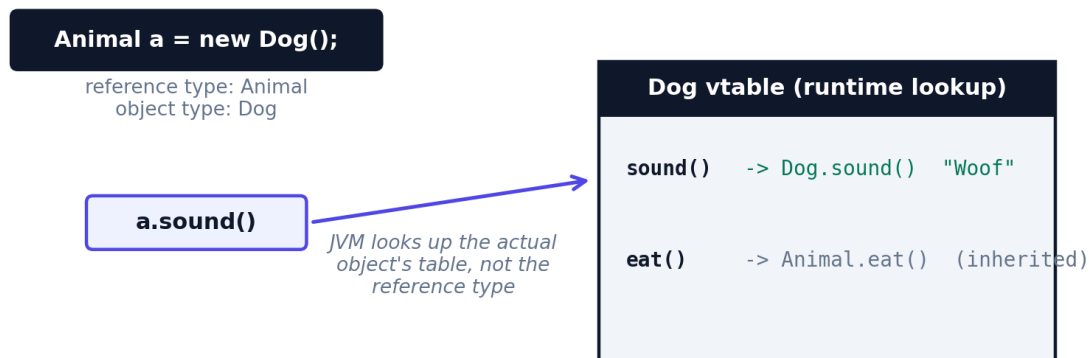
One overriding nuance interviewers probe: you may narrow the return type to a subtype (covariant return - parent returns Animal, child may return Dog), and you may widen the access modifier (protected to public) but never narrow it. Parameters must stay identical, or it becomes overloading.

9. Dynamic dispatch + the vtable

How does the runtime know which overridden method to call? Through dynamic dispatch, commonly implemented with a virtual method table (vtable). The JVM spec does not literally mandate a vtable, but in practice almost every JVM and C++ compiler uses a vtable-like dispatch table: each class with overridable methods has a table of pointers

to its actual method versions. Every object holds a hidden pointer to its class's vtable, so a call looks up the real method at runtime. That lookup is dynamic dispatch.

Dynamic dispatch: base reference, derived object -> derived method runs



Each object points to its class vtable; a virtual call follows the pointer to the correct overridden method.

C++ differs: C++ builds a vtable only for classes that have at least one 'virtual' method. Java builds dispatch in for all non-final, non-static methods automatically.

💡 **Aha - Vtable matlab har class ki ek phone directory jismein likha hai kaunsa method actually kahan hai. Object ke paas us directory ka number hota hai, toh call karte waqt sahi version dhoondh leta hai - yahi runtime pe sahi method chunna hai.**

Upcasting vs downcasting

Casting between a parent and child reference. Upcasting (child to parent reference) is always safe and is what enables runtime polymorphism. Downcasting (parent to child reference) is risky and can throw `ClassCastException`, so guard it with `instanceof` first.

Java

```
Animal a = new Dog();           // upcast - always safe
if (a instanceof Dog) {         // check before downcast
    Dog d = (Dog) a;            // safe downcast
}
```

💡 **Aha - Upcast matlab Dog ko Animal bol dena - hamesha safe, kyunki Dog toh Animal hai hi. Downcast matlab Animal ko wapas Dog banana - risky, pehle instanceof se check karo,**

warna ClassCastException.

10. Abstract class vs interface

Both let you define a contract that subclasses must fulfil, but they are not interchangeable.

Aspect	Abstract class	Interface (Java 8+)
Instantiate	No	No
Methods	Concrete + abstract mix	Abstract + default + static
Fields	Instance fields allowed	Only public static final constants
Constructor	Yes	No
How many	Extend only ONE	Implement MANY
Use when	Related classes share code + state	Unrelated classes share a capability

Abstract class vs Interface (Java 8+)

abstract class

- constructors: YES
- instance fields: YES (any state)
- concrete + abstract methods
- a class extends ONLY ONE
- use: share code among related classes

interface

- constructors: NO
- fields: public static final only
- default + static + abstract methods
- a class implements MANY
- use: common behavior, unrelated classes

Abstract class = shared code among related classes; interface = a capability unrelated classes can implement.

C++ differs: C++ has no 'interface' keyword. An interface is just an abstract class where every method is pure virtual (= 0). C++ also allows multiple inheritance of such classes, which is how it gets the same effect.

💡 **Aha - Abstract class matlab aadhi bani hui cheez jo related classes apna ke poori karti hain (ek hi parivaar). Interface matlab ek capability ka contract jo koi bhi class utha sakti hai - jaise 'Driveable' ko car bhi implement kare aur tractor bhi, bhale dono ka koi rishta na ho.**

11. Access modifiers

This is where Java and C++ differ the most, so know which one you are answering for. Java has FOUR levels (it adds a package level); C++ has THREE (no package idea at all). Same keywords, different rules.

Modifier	Java - visible from	C++ - visible from
private	Same class only	Same class (+ friends) only
default (no keyword)	Same package (package-private)	No such level - class members default to private, struct members to public
protected	Same package + subclasses	This class + derived classes (no package idea)
public	Everywhere	Everywhere

C++ differs: Two traps if you grew up on the other language. (1) C++ has **NO** package-private level - leave the keyword off and a class's members are private, a struct's are public. (2) C++ **protected** means this class plus derived classes only; it has nothing to do with packages, because C++ has no packages. Java's protected is wider: same package **OR** a subclass.

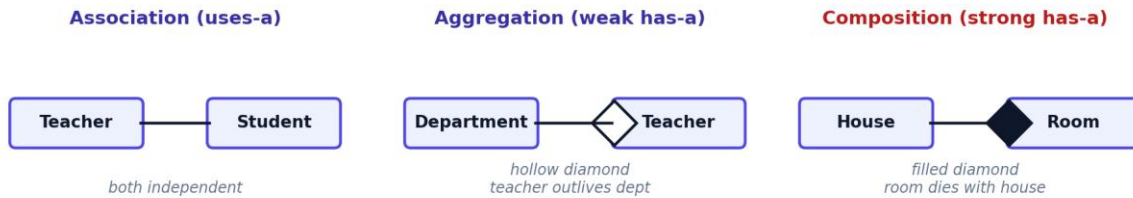
💡 **Aha - Access modifiers matlab ghar ke darwaze:** private apna bedroom (sirf tum), protected parivaar + bachche (subclasses), public main gate (sab aa sakte hain). Java mein ek extra darwaza hai - default/package, jo poore mohalle (same package) ke liye khula hai. C++ mein yeh mohalla wala darwaza hai hi nahi.

12. Association vs aggregation vs composition

These describe how objects relate. All three are has-a or uses-a links; the difference is ownership and lifetime.

Relationship	Strength	Lifetime rule	Example
Association	uses-a	Both independent	Teacher uses a Whiteboard
Aggregation	weak has-a	Child can outlive the parent	Department has Teachers
Composition	strong has-a	Child dies with the parent	Car has an Engine

Association vs Aggregation vs Composition (UML diamond on the owner)



Aggregation = hollow diamond (parts survive the owner); composition = filled diamond (parts die with the owner).

💡 **Aha - Aggregation matlab department aur teachers - department band ho jaaye toh teachers doosri jagah chale jaate hain, zinda rehte hain. Composition matlab car aur engine - car scrap ho gayi toh us car ka engine bhi gaya, child parent ke saath marta hai.**

13. Static vs dynamic binding + static members

Binding is when a method call is matched to its method body. Static (early) binding happens at compile time - overloaded methods, static methods, private and final methods. Dynamic (late) binding happens at runtime - overridden instance methods, resolved by the actual object.

Static members belong to the CLASS, not any object: one shared copy for all instances. A static variable counts every object created; a static method can be called without an object but cannot use instance fields.

Important nuance: static methods are HIDDEN, not overridden. A subclass static method with the same signature hides the parent's, and which one runs depends on the reference type, not the object.

💡 **Aha - Static member matlab classroom ka common blackboard - ek hi hai, sab students share karte hain, kisi ek student ka apna nahi. Dynamic binding matlab actual object dekh ke runtime pe sahi method chunna.**

Java is always pass-by-value

A classic trap. Java passes a COPY of the argument. For a primitive, that copy is the number. For an object, the copy is the reference (the address). So modifying the object's fields inside a method DOES affect the caller, but reassigning the parameter to a new object does NOT.

Java

```
void rename(Dog d) { d.name = "Bruno"; } // caller sees it
```

```
void reset(Dog d) { d = new Dog(); } // caller unaffected
```

💡 **Aha - Pass-by-value matlab tumne dost ko apne ghar ka address likh ke de diya. Woh us address pe jaake ghar ka rang badal de toh tumhe bhi dikhega (object field change). Par woh apni parchi pe naya address likh le toh tumhare ghar pe koi farak nahi (reassign).**

Object class methods (equals, hashCode, toString)

Every Java class silently extends Object, which gives equals() (default compares references), hashCode() (an int for hashing), and toString(). If you put your object in a HashMap or HashSet, override equals() and hashCode() together - the contract is that two objects equal by equals() must return the same hashCode().

💡 **Aha - equals aur hashCode hamesha jodi mein override karo. Agar do objects barabar hain (equals true) par unka hashCode alag hai, toh HashMap unhe alag dabbo mein daal dega aur tumhara lookup fail ho jaayega.**

Error vs exception

Both extend Throwable. An exception (like NullPointerException or a checked IOException) is something your code can catch and recover from. An error (like OutOfMemoryError or StackOverflowError) is a serious system failure you are not expected to catch.

💡 **Aha - Exception matlab woh problem jise tum sambhaal sakte ho (try-catch), jaise file nahi mili. Error matlab system hi gir gaya (memory khatam), usse pakad ke kuch fayda nahi.**

14. Shallow copy vs deep copy

Copy an object that holds a reference to another object, and you hit this. A shallow copy duplicates the top object but shares the inner referenced objects - change the inner one through either copy and both see it. A deep copy duplicates the inner objects too, so the copies are fully independent.

💡 **Aha - Shallow copy matlab tumne dost ko apne Google Doc ka same link de diya - woh edit kare toh tumhare paas bhi badal jaata hai. Deep copy matlab tumne poori file ki alag copy banaake di - ab woh apni copy badle toh tumhari original safe.**

15. Exception handling essentials (Java)

An OOP round almost always slips in exception questions, so know this cold. Everything below is Throwable's family: Throwable splits into Error (serious system failure you do not catch) and Exception (your code's problem, which you can catch and recover from).

Checked vs unchecked exceptions

Checked exceptions are checked by the compiler: you must either catch them or declare them with throws, or the code will not compile (IOException, SQLException). They model recoverable outside problems like a missing file. Unchecked exceptions extend RuntimeException and are not compiler-checked; they signal bugs in your logic (NullPointerException, ArrayIndexOutOfBoundsException, IllegalArgumentException). Rule of thumb: checked for problems the caller can sensibly handle, unchecked for programmer mistakes.

throw vs throws

- throw - a statement that actually raises an exception right now: `throw new IllegalArgumentException("age < 0")`.
- throws - a clause in the method signature that declares which checked exceptions a method MIGHT raise, passing the duty to handle to the caller: `void read() throws IOException`.
- One letter apart, opposite jobs: throw fires it, throws warns about it.

try - catch - finally, and can finally be skipped?

try wraps risky code, catch handles a matching exception type, and finally runs no matter what - exception or not, return or not - so it is where you close files and release locks. finally runs even when the try or catch block hits a return. The only ways finally does NOT run: the JVM itself stops (`System.exit()`), the thread is killed or the machine dies, or the try/catch never finishes because it hits an infinite loop. So for normal control flow, treat finally as guaranteed.

Java

```
try {
    return readFile();           // even with this return...
} catch (IOException e) {
    return fallback();
} finally {
    closeHandle();              // ...finally still runs first
}
// Modern way - try-with-resources auto-closes, no finally needed:
try (BufferedReader r = new BufferedReader(...)) {
    return r.readLine();
} // r.close() called automatically
```

Custom exceptions

Make your own by extending Exception (checked) or RuntimeException (unchecked). Use them to give a domain-specific name and carry extra data, which reads far better than a generic exception.

Java

```
class InsufficientBalanceException extends RuntimeException {
    InsufficientBalanceException(String msg) { super(msg); }
}
```

```

}
if (amount > balance)
    throw new InsufficientBalanceException("Need " + amount);

```

C++ differs: C++ has no checked/unchecked split - every exception is effectively unchecked, and you catch by value or reference with `catch (const std::exception& e)`. There is no finally; C++ uses RAII (destructors free resources when scope ends) instead, which is why Java's try-with-resources feels like RAII bolted on.

💡 Aha - throw matlab ab gend phenk di (exception raise), throws matlab pehle warning board laga diya 'yahan gend aa sakti hai' (method declare). finally woh safai wala hai jo show ke baad aata hi aata hai, chahe match jeeto ya haaro.

16. Java Collections crash course

For Java placements this gets asked as much as the pillars. You do not need every class - know the four big comparisons and what backs each structure.

ArrayList vs LinkedList

Aspect	ArrayList	LinkedList
Backed by	A resizable array	A doubly linked list
Random access (get i)	O(1)	O(n) - must walk
Insert/delete in middle	O(n) - shift elements	O(1) once you hold the node
Memory	Less (just the array)	More (two pointers per node)
Use when	Mostly reads / index access	Many inserts/deletes at ends

HashMap vs Hashtable

Aspect	HashMap	Hashtable
Thread-safe	No	Yes (every method synchronized)
Null keys/values	One null key, many null values	No nulls at all
Speed	Faster (no locking)	Slower (locks on every call)
Status	Use this	Legacy - prefer ConcurrentHashMap for threads

Follow-up they love: for thread-safe maps today, do not reach for Hashtable - use ConcurrentHashMap, which locks only small segments instead of the whole map, so it scales far better. One catch: ConcurrentHashMap allows no null keys or values at all, unlike HashMap, so check your code before swapping one for the other.

What backs each structure

- HashSet is backed by a HashMap - each element is stored as a key with a dummy constant value, which is why a HashSet needs the same equals() and hashCode() discipline a HashMap key needs.
- LinkedHashMap / LinkedHashSet keep insertion order on top of the hash structure.
- TreeMap / TreeSet are backed by a red-black tree, so they keep keys sorted and give $O(\log n)$ operations instead of $O(1)$.

Why override hashCode() (and equals())

Hash-based collections find an object in two steps: hashCode() picks the bucket, then equals() confirms the exact match inside that bucket. If you override equals() to compare fields but leave the default hashCode(), two equal objects can land in different buckets, so a get() or contains() misses an object that is logically present. That is why the rule is to override both together, and the contract is: equal objects MUST share a hashCode.

💡 **Aha - hashCode pehle batata hai object kaunse dabbe (bucket) mein hai, phir equals confirm karta hai wahi object hai. Agar dono ko jodi mein override nahi kiya, toh barabar objects alag dabbo mein chale jaate hain aur HashMap unhe dhoondh hi nahi paata.**

17. Why String is immutable + the String pool

String is final and immutable: once created, its characters never change. Any method that looks like it edits a String (concat, replace) actually returns a brand-new String. This is an interview favourite, and the why has four parts.

- Security - Strings hold sensitive values like file paths, URLs, and usernames passed into APIs. If a String could change after a security check, an attacker could swap it between the check and the use.
- Hashing - String is the most common HashMap key. Immutability means its hashCode can be computed once and cached, and a key can never change bucket after it is stored.
- Thread safety - an immutable object can never reach a bad state, so many threads share one String with zero locking.
- String pool - because Strings cannot change, the JVM keeps a pool of string literals and reuses one object for identical literals, saving memory. That reuse is only safe because no one can mutate a shared String.

Follow-up: if you need a changing string in a loop, use StringBuilder (not thread-safe, fast) or StringBuffer (thread-safe, slower). Building a long string with + in a loop creates a new String each time and is $O(n^2)$.

C++ differs: C++ std::string is mutable - you can change characters in place and there is no string pool. This is a frequent mix-up: the immutability story is a Java (and Python) thing, not C++.

💡 **Aha - String immutable matlab ek baar likh diya toh patthar pe lakeer. Badalna ho toh nayi String banti hai, purani waise hi rehti hai. Isi liye woh HashMap ki sabse safe chaabi hai aur pool mein reuse ho jaati hai.**

18. Beyond the basics (advanced, not all fresher-required)

SOLID and design patterns below are senior-round and good-design material. For most fresher interviews they are recognise-the-words and one-line-definition, not memorise-every-pattern. The ONE exception worth knowing properly: the Singleton pattern and the five SOLID names with a one-liner each. Read the rest once so the terms are not strangers.

19. SOLID principles

Five design principles for code that is easy to change. Know each as a one-liner.

SOLID - five design principles for maintainable OOP



The five SOLID principles at a glance.

- S - Single Responsibility: a class should have one reason to change (one job).
- O - Open/Closed: open for extension, closed for modification - add new behaviour without editing tested code.
- L - Liskov Substitution: a subclass must be usable wherever its base class is, without breaking behaviour.
- I - Interface Segregation: many small focused interfaces beat one fat one; clients should not depend on methods they do not use.
- D - Dependency Inversion: depend on abstractions, not concrete classes.

💡 **Aha - SOLID ko aise yaad rakho:** har class ka ek hi kaam (S), naya feature add karo bina purana chheda (O), bachcha class parent ki jagah bina tode chal jaaye (L), chhote chhote interface (I), aur depend karo abstraction pe na ki concrete class pe (D).

Cohesion vs coupling

The two words that summarise good design, often asked right after SOLID. Cohesion is how focused one class is (do its methods all serve one job?). Coupling is how dependent classes are on each other. The goal is high cohesion and low coupling.

💡 **Aha - High cohesion matlab ek class ka ek hi clear kaam ho, idhar-udhar ka sab kuch ek jagah na thusa ho. Low coupling matlab classes ek doosre pe kam depend karein, taaki ek badlo toh doosri na toote.**

20. Design patterns (recognise the words)

Reusable solutions to common design problems. For a fresher round, one line each is enough.

- Singleton - exactly one instance shared everywhere (a config or logger). Watch the thread-safety nuance.
- Factory - a method decides which subclass object to create, so callers do not use new directly.
- Observer - subjects notify a list of subscribers on change (event listeners, pub-sub).
- Strategy - swap an algorithm at runtime by composing a behaviour object instead of subclassing.

Mindset warning for Singleton: if your first instinct is Singleton, it is probably wrong. It is the most over-used pattern and acts like a global variable. Reach for it only for genuinely single resources, and make it thread-safe.

💡 **Aha - Singleton matlab ek hi banda poore office ka HOD - sirf ek instance. Factory matlab tum order do 'ek chai', kitchen decide karti hai kaunsi banani hai. Observer matlab WhatsApp group - ek ne message bheja, sab subscribers ko mil gaya.**

21. Java vs C++ - the OOP differences

Aspect	Java	C++
Multiple class inheritance	Not allowed (use interfaces)	Allowed (virtual inheritance for the diamond)
Virtual methods	All instance methods virtual by default	Must mark 'virtual' explicitly
Memory	Garbage collector	Manual (new / delete) or smart pointers
Destructor	None (GC + close())	Explicit ~ClassName()
Interface keyword	Yes	No (pure virtual abstract class)
Operator overloading	No	Yes
Pointers	References only	Pointers + references

💡 **Aha - Java vs C++ ek line mein: Java tumhare liye memory sambhaalta hai aur har method virtual hai (safe but kam control). C++ tumhe poora control deta hai - manual memory, explicit**

virtual, multiple inheritance - power zyada, paer pe kulhaadi bhi aasaan.

22. Comparison tables (the ones interviewers love)

Overloading vs Overriding

Aspect	Overloading	Overriding
Binding	Compile-time (static)	Runtime (dynamic)
Where	Same class	Parent and child class
Signature	Different parameter list	Same signature
Inheritance	Not needed	Required

Abstraction vs Encapsulation

Aspect	Abstraction	Encapsulation
Hides	Complexity (the how)	Data (the state)
Level	Design	Implementation
Achieved by	Abstract classes, interfaces	private + getters and setters

Aggregation vs Composition

Aspect	Aggregation	Composition
Strength	Weak has-a	Strong has-a
Child lifetime	Survives the parent	Dies with the parent
Example	Department has Teachers	Car has an Engine

23. Interview triggers - if they ask X, say Y

If they ask / hint	Reach for
Difference between abstraction and encapsulation	Abstraction hides complexity (design); encapsulation hides data (implementation)
Same method name, different arguments	Overloading - compile-time
Child changes parent's method	Overriding - runtime, via the vtable
When abstract class vs interface	Shared code + state = abstract; capability across unrelated classes = interface
Car and engine relationship	Composition - engine dies with the car
Only one instance allowed	Singleton (mention thread-safety)
Add features without touching old code	Open/Closed principle
Why no multiple inheritance in Java	Diamond problem - Java uses interfaces instead
How does runtime pick the method	Dynamic dispatch through the vtable
Two objects share an inner object after copy	Shallow copy - needs a deep copy

24. Common mistakes + traps

Wrong thinking	Correct thinking
Abstraction and encapsulation are the same	Abstraction hides complexity; encapsulation hides data - different concerns
Overloading is runtime	Overloading is compile-time; only overriding is runtime
Java supports multiple class inheritance	Java forbids it for classes; use multiple interfaces
An interface can have instance fields	Interface fields are public static final constants only
You can instantiate an abstract class	No - it must be subclassed first
Static methods can be overridden	They are hidden, not overridden - resolved by reference type
Composition and aggregation are the same	Composition = child dies with parent; aggregation = child survives
Singleton is always a good idea	It is the most over-used pattern; often a global in disguise
Private members are inherited and usable	They are inherited but not directly accessible in the child

25. Model interview answers - say it like this

The 45-question bank below is for rapid recall. This section is the opposite: the 19 questions an OOP round actually opens with, each with a full answer you can speak in 20 to 40 seconds. Every answer follows the same shape - define it in one line, give one concrete example, then handle the follow-up before they ask. Read these out loud once; that is what makes them stick.

How to use a model answer: lead with the one-line definition so the interviewer knows you know it, then ground it in ONE example (not three), then stop. Rambling past the example is how strong answers turn average. The follow-up line is the question they ask next - have it ready and you look two steps ahead.

Q. What is OOP, and why use it instead of plain functions?

Say this: OOP models a program as objects, where each object bundles its data with the methods that act on that data. Instead of free functions passing loose data around, the object that owns the data also owns the behaviour. The payoff is real-world modelling, reuse through inheritance, and easier maintenance, because each piece is self-contained, so you change one class without breaking the rest. The four pillars that make it work are encapsulation, abstraction, inheritance, and polymorphism.

Follow-up they hit you with: Give one concrete example, like a BankAccount that keeps balance private and only changes it through deposit() and withdraw(), so no outside code can force a negative balance.

Q. Explain the four pillars with one real example.

Say this: Take a payment app. Encapsulation: the Wallet keeps balance private and exposes deposit() and withdraw(), so nobody sets it directly. Abstraction: a Payment interface exposes pay(), and the caller never sees whether it is UPI or card underneath. Inheritance: UpiPayment and CardPayment both extend a base Payment and reuse its common code.

Polymorphism: you call `pay()` on a `Payment` reference and the right version runs based on the actual object. One example, all four pillars.

Follow-up they hit you with: *They often ask which pillar matters most. Say none alone - encapsulation and abstraction protect the design, inheritance and polymorphism give the flexibility, and they only pay off together.*

Q. Abstraction vs encapsulation - the difference?

Say this: Both hide something, but different things at different levels. Encapsulation hides DATA: you make fields private and force access through getters and setters, so the object can never reach a bad state. That is an implementation-level concern. Abstraction hides COMPLEXITY: you expose only what the caller needs through an interface or abstract class and keep the how invisible. That is a design-level concern. Short version: encapsulation hides the data, abstraction hides the implementation detail.

Weak answer (skip it): *Saying they are the same, or that both just hide data. That throws away the whole question.*

Follow-up they hit you with: *Can you have one without the other? Yes. A public field behind a clean interface has abstraction but no encapsulation; a private field exposed through a getter that dumps everything has encapsulation but weak abstraction.*

Q. Overloading vs overriding?

Say this: Same method name, two different mechanisms. Overloading is the same name with different parameter lists in the same class, and the compiler picks which one at compile time, so it is compile-time polymorphism. Overriding is a subclass redefining a parent method with the exact same signature, and the runtime picks the version from the actual object, so it is runtime polymorphism. Overloading needs no inheritance; overriding requires it.

Weak answer (skip it): *Calling overloading runtime. It is compile-time, and that one slip sinks the answer.*

Follow-up they hit you with: *Can the return type alone tell two overloaded methods apart? No. The compiler resolves on the parameter list, not the return type, so changing only the return type is a compile error.*

Q. When overriding, can you change the return type or the access level?

Say this: Yes, within limits, and this is the covariant-return rule. You may narrow the return type to a subtype of the parent's return type, so if the parent returns `Animal`, the override may return `Dog`. You may also widen the access modifier, protected to public, but never narrow it, because that would break callers holding a parent reference. The parameter list must stay identical; change it and it stops being an override and becomes an overload.

Follow-up they hit you with: *Can the override throw broader checked exceptions? No. It may throw the same or narrower checked exceptions, or none, but not new broader ones, for the same substitutability reason.*

Q. Abstract class vs interface - when do you pick which?

Say this: Use an abstract class when related classes share both code and state and there is a clear is-a hierarchy, because it can hold instance fields, constructors, and concrete methods, and a class extends only one. Use an interface when unrelated classes share a capability, because it is a contract any class can implement, and a class implements many. Rule of thumb: shared code and state, abstract class; a shared capability across unrelated types, interface.

Follow-up they hit you with: *Since Java 8 interfaces have default methods, so why keep abstract classes? Because interfaces still cannot hold instance state or constructors, and you extend only one class. Java 9 even added private interface methods, but still no instance fields or constructors, so abstract classes own the shared-state case.*

Q. Why doesn't Java support multiple class inheritance?

Say this: To avoid the diamond problem. If a class could inherit from two classes that both descend from a common ancestor, it could get two versions of the same member and the compiler would not know which one wins. Java forbids multiple class inheritance to kill that ambiguity, and gives you multiple interfaces instead, because interfaces hold no instance state or constructors, so there is no init order to fight over. When two interfaces provide the same default method, Java forces you to override it and resolve the clash yourself.

Follow-up they hit you with: How does C++ handle it? C++ allows multiple inheritance and solves the diamond with virtual inheritance, so only one shared copy of the base exists.

Q. How does runtime polymorphism actually work under the hood?

Say this: Through dynamic dispatch, commonly implemented with a virtual method table, the vtable. The spec does not literally require one, but in practice each class with overridable methods gets a table of pointers to its actual method versions, and every object carries a hidden pointer to its class vtable. When you call an overridden method through a parent reference, the runtime follows the object's vtable pointer to the real method, so the actual object decides, not the reference type. In Java every non-final instance method is virtual by default; in C++ you mark it virtual or the call binds to the reference type at compile time.

Follow-up they hit you with: What does it cost? One extra pointer indirection per virtual call, and the method is harder to inline, which is exactly why C++ makes virtual opt-in.

Q. Static vs dynamic binding?

Say this: Binding is matching a method call to its body. Static or early binding happens at compile time and covers overloaded, static, private, and final methods, because the compiler already knows the exact target. Dynamic or late binding happens at runtime and covers overridden instance methods, resolved by the actual object through the vtable. So overloading is static binding and overriding is dynamic binding.

Follow-up they hit you with: Why are static and private methods statically bound? Because they cannot be overridden, so there is no runtime ambiguity - the target is fixed at compile time.

Q. Can you override static, private, or final methods?

Say this: No to all three, for different reasons. A static method belongs to the class, not an object, so a subclass version with the same signature hides it rather than overrides it, and which one runs depends on the reference type. A private method is not visible to the subclass at all, so a same-named method there is just a brand-new method. A final method is explicitly locked against overriding by the compiler. Only non-static, non-final, visible instance methods can be overridden.

Follow-up they hit you with: What is method hiding then? It is the static-method case: the subclass method hides the parent's, resolved by the reference type at compile time, not by the object at runtime.

Q. Why must you override equals() and hashCode() together?

Say this: Every class inherits both from Object. equals() by default compares references, and hashCode() returns an int used by hash-based collections. The contract is that two objects equal by equals() must return the same hashCode(). So if you override equals() to compare fields, you must override hashCode() to match, otherwise a HashMap or HashSet puts equal objects in different buckets and your lookups fail.

Weak answer (skip it): Overriding only equals() and leaving hashCode() default. It compiles fine and silently breaks HashMap lookups.

Follow-up they hit you with: Does equal hashCode imply equal objects? No. Different objects can share a hashCode, that is a collision, which is why hashCode narrows the bucket and equals() makes the final call.

Q. Association vs aggregation vs composition?

Say this: All three describe how objects relate; the difference is ownership and lifetime. Association is a plain uses-a link with no ownership, like a Teacher using a Whiteboard. Aggregation is a weak has-a where the part can outlive the whole, like a Department that has Teachers - close the department and the teachers move on. Composition is a strong has-a where the part dies with the whole, like a Car built with an Engine - scrap the car and that engine is gone.

Follow-up they hit you with: In UML, aggregation is a hollow diamond and composition is a filled diamond, and the deciding test is always the lifetime question: can the part exist without the whole?

Q. Is Java pass-by-value or pass-by-reference?

Say this: Java is always pass-by-value. For a primitive, the copied value is the number itself. For an object, the copied value is the reference, the address, not the object. So inside a method you can change the object's fields and the caller sees it, because both references point to the same object, but if you reassign the parameter to a new object, the caller is unaffected, because you only changed the local copy of the reference.

Weak answer (skip it): *Saying objects are passed by reference. The reference is passed by value, and a reassignment inside the method proves it.*

Follow-up they hit you with: *Prove it: a method that sets a field changes the caller's object, but a method that does param = new Object() leaves it untouched.*

Q. Shallow copy vs deep copy?

Say this: When an object holds a reference to another object and you copy it, this matters. A shallow copy duplicates the outer object but shares the inner referenced objects, so changing an inner object through either copy affects both. A deep copy duplicates the inner objects too, so the two copies are fully independent. Java's `Object.clone()` is shallow and needs the class to implement `Cloneable`; for a deep copy you override `clone()`, call `super.clone()`, then clone every mutable field yourself, or use a copy constructor, or deep-copy through serialization.

Follow-up they hit you with: *When does a shallow copy actually bite you? Only when the shared inner object is mutable; if it is immutable like a String, sharing it is safe and shallow is fine.*

Q. Walk me through SOLID.

Say this: Five principles for code that is easy to change. Single Responsibility: a class has one reason to change. Open/Closed: open to extension, closed to modification, so you add behaviour without editing tested code. Liskov Substitution: a subclass must work anywhere its base class does, without breaking expectations. Interface Segregation: many small focused interfaces beat one fat one. Dependency Inversion: depend on abstractions, not concrete classes.

Follow-up they hit you with: *Give one violation, like a class that both parses a file and emails a report - that breaks Single Responsibility, because two unrelated reasons could force it to change.*

Q. What does 'composition over inheritance' mean, and why prefer it?

Say this: It means build behaviour by holding other objects, has-a, rather than deep inheritance trees, is-a. Inheritance is rigid: it is fixed at compile time, exposes the parent's internals to the child, and a tall hierarchy gets brittle, where one change high up breaks everything below. Composition lets you swap the held object, even at runtime, and keeps classes loosely coupled. Use inheritance only for a genuine is-a relationship; otherwise compose.

Follow-up they hit you with: *A classic example: instead of a GoldUser extends User extends Account tower, give User a Membership object you can swap, so changing tier is a field change, not a new subclass.*

Q. Is the Singleton pattern a good idea?

Say this: It has its place, but it is the most over-used pattern, so I am careful with it. It guarantees one instance and a global access point, which fits genuinely single resources like a config holder or a logger. The downside is that it behaves like a global variable, hides dependencies, and makes testing hard because you cannot easily swap it. So I use it only for truly single resources, make it thread-safe, and otherwise prefer passing the dependency in.

Follow-up they hit you with: *How do you make it thread-safe? The cleanest ways are an enum singleton or a static holder class. Both are thread-safe for free because the classloader initializes a class exactly once and single-threaded, so you never hand-write double-checked locking, which is easy to get wrong on memory visibility.*

Q. Difference between `this()` and `super()`, and what is constructor chaining?

Say this: Both call constructors and both must be the first statement in a constructor. `super()` calls a parent-class constructor; if you do not write it, the compiler inserts a no-arg `super()` for you, so the parent is always built first. `this()`

calls another constructor in the SAME class, which is constructor chaining: you funnel several overloaded constructors into the one that holds the real init logic, so you write that logic once. You cannot use `this()` and `super()` in the same constructor, because both want the first line.

Follow-up they hit you with: What if the parent has no no-arg constructor? Then the child must call `super(args)` explicitly; otherwise the compiler-inserted no-arg `super()` finds no match and you get a compile error.

Q. How do you design an immutable class?

Say this: Four rules. Make the class final so no subclass can weaken it. Make every field private and final. Set all fields once in the constructor and add no setters. And for any mutable field, like a Date or a List, store a defensive copy in the constructor and return a defensive copy from the getter, so no outside reference can reach in and mutate the internals. String is the standard example.

Follow-up they hit you with: Why is an immutable object automatically thread-safe? Because its state never changes after construction, so threads can share it freely with no locking and no visibility issues.

26. Rapid-fire FAQ bank - 45 quick-recall questions

One or two lines each, for fast revision once the model answers above are solid. If you can fire these back out loud, you are ready.

Question	Answer
What is OOP?	Modelling a program as objects that bundle data with the behaviour acting on it.
Class vs object?	A class is the blueprint; an object is one instance built from it.
Name the four pillars.	Encapsulation, abstraction, inheritance, polymorphism.
What is encapsulation?	Bundling data and methods into one unit and controlling access via private + getters and setters.
What is abstraction?	Hiding complexity and exposing only the essentials, via abstract classes and interfaces.
Abstraction vs encapsulation?	Abstraction hides complexity (design level); encapsulation hides data (implementation level).
What is inheritance?	A child class reusing fields and methods of a parent, modelling an is-a relationship.
What is polymorphism?	One name behaving differently by object or arguments; compile-time (overloading) and runtime (overriding).
Overloading vs overriding?	Overloading = same name, different parameters, compile-time; overriding = same signature in a subclass, runtime.
Is overloading runtime or compile-time?	Compile-time (static binding).
What gives runtime polymorphism?	Overriding, resolved by the actual object via the vtable.
What is a vtable?	A per-class table of method pointers an object uses to find the right overridden method at runtime.
Static vs dynamic binding?	Static is resolved at compile time (overloading, static, final); dynamic at runtime (overriding).
Abstract class vs interface?	Abstract class has state + concrete code and you extend one; interface is a capability and you implement many.
Can an interface have methods	Yes since Java 8 - default and static methods.

with bodies?	
Can you instantiate an abstract class?	No; it must be subclassed.
What is the diamond problem?	Ambiguity when a class inherits the same member through two paths from a common ancestor.
How does Java avoid the diamond problem?	It forbids multiple class inheritance; conflicting interface defaults must be overridden.
Single vs multilevel inheritance?	Single = one parent; multilevel = a chain (C from B from A).
What is a constructor?	A special method that initialises a new object; same name as the class, no return type.
Default vs parameterised constructor?	Default takes no args (compiler-provided if none written); parameterised takes values.
Difference between this and super?	this is the current object; super is the parent (super() calls the parent constructor).
Does Java have destructors?	No; the garbage collector frees memory, and you close resources explicitly.
Association vs aggregation vs composition?	Association = uses-a; aggregation = weak has-a (child survives); composition = strong has-a (child dies with parent).
Example of composition?	Car has an Engine - destroy the car and that engine is gone.
Example of aggregation?	Department has Teachers - close the department and teachers move on.
What are access modifiers? (Java)	private (class), default (package), protected (package + subclasses), public (everywhere). C++ has only private/protected/public - no package level.
What is a static member?	A field or method that belongs to the class, shared by all objects.
Are static methods overridden?	No - they are hidden, resolved by the reference type.
Shallow vs deep copy?	Shallow shares inner referenced objects; deep duplicates them so copies are independent.
What is method hiding?	A subclass static method with the same signature hiding the parent's.
Can a constructor be inherited?	No; constructors are not inherited, but a child calls the parent's via super().
Can you override a final method?	No; final methods cannot be overridden.
Can you overload the main method?	Yes; only the standard signature is the entry point.
What is an abstract method?	A method declared without a body that subclasses must implement.
Expand SOLID.	Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion.
What is the Single Responsibility Principle?	A class should have one reason to change - one job.
What is the Liskov Substitution Principle?	A subclass must be usable wherever its base class is, without breaking behaviour.
What is the Singleton pattern?	Ensures exactly one instance of a class, shared everywhere.
What is the Factory pattern?	A method decides which subclass to instantiate, hiding new from the caller.
What is the Observer pattern?	A subject notifies a list of subscribers when its state changes.
What is the Strategy pattern?	Swapping an algorithm at runtime by composing a behaviour object.
Composition over inheritance - why?	Composition is more flexible and avoids deep brittle hierarchies; prefer has-a over is-a when unsure.
Is Java pass-by-value or reference?	Always pass-by-value; for objects, the value passed is the reference.
Why is multiple inheritance risky?	The diamond problem - ambiguous members from a shared ancestor.

27. The 30-minute revision card

Last 30 minutes before the round. Read only this.

The four pillars, one line each:

- Encapsulation - bundle data + methods, control access (private + getters/setters).
- Abstraction - hide complexity, show essentials (abstract classes, interfaces).
- Inheritance - reuse via is-a.
- Polymorphism - one name, many forms.

The traps that score points:

- Abstraction hides complexity (design); encapsulation hides data (implementation).
- Overloading = compile-time; overriding = runtime.
- Static methods are hidden, not overridden.
- Java forbids multiple class inheritance; C++ allows it (virtual inheritance for the diamond).
- Composition = child dies with parent; aggregation = child survives.

Abstract class vs interface, one line:

- Abstract class - shared code + state, extend ONE.
- Interface - a capability, implement MANY, fields are constants only.

SOLID in five words:

- One job, extend-don't-modify, substitutable subclass, small interfaces, depend on abstractions.

Two traps that cost you easy points:

- Calling overloading runtime - it is compile-time.
- Saying abstraction and encapsulation are the same - they are not.

28. Final word

You have the four pillars with real examples, overloading vs overriding straight, abstract vs interface clear, and SOLID in one line each. An OOP round often opens with the pillars, one polymorphism question, and one design question. You have already practised all three. Answer calmly and you will have cleared the OOP portion of the round.